

Boosting Cloud Resource Efficiency: Workload Prediction Using XGBoost for Auto-Scaling

1st João Victor Tavares Esteves

Departamento de Engenharia de Computação
Universidade Federal do Ceará
Fortaleza, Brazil
joaovtesteves2002@alu.ufc.br

2nd Gabriel Pinheiro Palitot Pereira

Departamento de Engenharia de Computação
Universidade Federal do Ceará
Fortaleza, Brazil
gabrielppalitot@alu.ufc.br

3rd Rodrigo da Silva Carvalho Maia

Departamento de Engenharia de Computação
Universidade Federal do Ceará
Fortaleza, Brazil
rodrigomaia@alu.ufc.br

Abstract—Cloud computing demands proactive resource scaling to ensure Quality of Service (QoS) and compliance with Service Level Agreements (SLA). This article revisits a prior study on workload prediction for auto-scaling cloud applications, which employed Support Vector Machines (SVM) for workload classification. In contrast, we propose the use of XGBoost, a gradient boosting algorithm, for enhanced prediction accuracy. Using the same real-world cloud dataset, our results demonstrate that XGBoost significantly outperforms SVM, achieving an Accuracy of 98.23% and a Balanced Accuracy of 85.53%, alongside substantial improvements in Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). These findings highlight the potential of XGBoost to provide more accurate and scalable workload predictions, making it a robust choice for proactive auto-scaling in cloud environments.

Index Terms—Machine Learning, XGBoost, Cloud Computing, Workload Prediction, Auto-scaling, SLA, QoS.

I. INTRODUCTION

The widespread adoption of cloud computing has revolutionized resource provisioning, enabling users to access software, platforms, and infrastructure on demand over the Internet. A key enabler of this flexibility is *elasticity*, which dynamically adjusts resources to match fluctuating workloads [3], [4]. Elasticity ensures that *Quality of Service* (QoS) levels are maintained [5], helping to meet the expectations outlined in *Service Level Agreements* (SLA) between cloud providers and users [6].

Cloud workloads often experience significant variations due to changes in user demand and application behavior. Managing these fluctuations effectively requires *scalable resource allocation mechanisms*. While *reactive auto-scaling* adjusts resources based on current workload conditions, it often introduces delays that can lead to SLA violations and degraded QoS [8]. To address this limitation, *proactive auto-scaling* predicts future workloads and adjusts resources in advance, ensuring smooth and efficient cloud operations [7], [9].

Machine learning (ML) has emerged as a robust solution for proactive workload prediction, leveraging historical data

to forecast future resource demands and optimize cloud performance. Various ML techniques, including *Support Vector Machines* (SVM) [10] and *deep learning models* [11], have demonstrated success in workload forecasting. However, accurately predicting workloads across imbalanced classes, such as low or medium demand levels, remains a challenge.

To address these challenges, this paper explores the use of *XGBoost*, a gradient boosting algorithm known for its efficiency and superior performance on high-dimensional datasets [12]. Compared to SVM, XGBoost is designed to minimize errors and handle complex data distributions, making it a promising choice for cloud workload prediction [13]. Using the same real-world dataset evaluated in [2], our experimental results show that XGBoost achieves substantial improvements over SVM in metrics such as *Accuracy*, *Balanced Accuracy*, *Mean Absolute Error* (MAE), and *Root Mean Squared Error* (RMSE).

The remainder of this paper is structured as follows: Section II presents the background on workload prediction and auto-scaling. Section III discusses related work on ML-based workload forecasting. Section IV introduces the proposed XGBoost model. Section V describes the experimental setup, including dataset details and evaluation metrics. Section VI presents and discusses the results, comparing XGBoost with SVM. Finally, Section VII concludes the paper and highlights future research directions.

II. BACKGROUND

A. Cloud Computing, Elasticity and Auto-Scaling

Cloud computing can be defined as a new paradigm for hosting and delivering services over the internet [14]. It operates on a multi-layered architecture that abstracts physical resources to provide scalable services [14].

The cloud architecture infrastructure can be divided into three main layers [1]: the infrastructure layer, responsible for physical hardware; the virtualization layer, which partitions physical resources into virtual machines, ensuring greater

availability and reliability of IT services [15]; and finally, the service layer, which provides solutions such as IaaS, PaaS, and SaaS, addressing various user demands via the internet [16].

Virtualization is essential for cloud elasticity, as it enables the provisioning and deactivation of resources on demand. For example, virtual machines can be quickly instantiated to handle workload peaks, ensuring operational efficiency [15].

Elasticity is the ability of a system to dynamically add or remove resources (such as CPU cores, memory, virtual machine instances, and containers) to adapt to workload variations in real time. This characteristic is fundamental in cloud computing, as it allows automatic resource adjustment as demand fluctuates [17].

For elasticity to be effective, resource allocation and deallocation in the cloud must occur automatically and efficiently, without human intervention. Over-provisioning can lead to resource waste and additional costs, while under-provisioning can compromise system performance and violate service level agreements (SLA). To ensure quality of service (QoS), cloud computing employs a mechanism known as auto-scaling, which dynamically adjusts resources based on demand [18].

There are two main scaling models: vertical scaling and horizontal scaling. Vertical scaling (scale-up) involves increasing or decreasing the capacity of existing resources, such as adding more CPU, memory, or storage to a single machine. However, this approach is limited by the maximum capacity of the underlying hardware. On the other hand, horizontal scaling (scale-out) involves adding or removing server or container instances, distributing the workload more efficiently [19].

Additionally, auto-scaling can be classified into two categories. Reactive auto-scaling operates after detecting workload changes, meaning scaling decisions are triggered based on predefined metrics [20]. In contrast, proactive auto-scaling attempts to allocate resources in advance, predicting future demands based on historical data and adjusting resources before workload increases [19].

B. Workload Prediction

The concept of workload refers to the volume and type of processing that a computing system must execute within a given period. In the cloud computing context, a workload may include various tasks, such as application execution, data processing, database queries, user requests on a website, and other computational activities that require resources such as CPU, memory, storage, and bandwidth [24].

Workload prediction is essential in cloud computing as it enables efficient resource allocation, ensuring service quality and optimizing operational costs [21]. By anticipating future system demand, cloud providers can proactively adjust available resources to meet user needs. Effective prediction strategies allow for dynamic resource management, optimizing performance and enhancing user satisfaction. Advanced methods, such as deep neural networks and machine learning models, have shown great potential in improving prediction accuracy and resource allocation efficiency in cloud environments [22].

However, workload prediction presents several challenges. One of the main obstacles is the high volatility of cloud workloads, which have variable patterns, making it difficult for traditional forecasting methods to achieve precise predictions. Additionally, due to the diversity of applications, workload forecasting must consider the specific needs and characteristics of each service, making the process even more complex [23]. To address these challenges, several studies explore strategies based on machine learning, deep learning, and statistical methods to improve workload prediction accuracy [23].

C. Machine Learning and Workload Prediction

Machine learning (ML) has become a cornerstone in cloud computing, enabling intelligent resource management and workload prediction [25]. By analyzing historical data, ML models can identify patterns and trends, allowing cloud systems to anticipate future demands and optimize resource allocation.

Several machine learning techniques are being used for workload prediction, each with distinct characteristics and challenges. One of the most studied approaches is the use of deep learning models, including Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), Deep Belief Networks (DBN), and Autoencoders. These techniques are particularly effective in modeling temporal patterns, enabling more accurate predictions by capturing complex relationships in resource usage data [26].

Recurrent Neural Network (RNN)-based models, such as LSTM and GRU, are widely used for time series forecasting as they can maintain an internal state that allows capturing long-term dependencies in data sequences. In the context of workload prediction, these networks are trained with historical resource usage data and can anticipate future fluctuations, improving the efficiency of auto-scaling systems. However, despite their potential, these models require large volumes of training data and high computational capacity, which can hinder their large-scale implementation [26].

It is challenging to find studies that apply a technique called XGBoost. XGBoost (Extreme Gradient Boosting) is a machine learning system based on decision trees, designed to be highly scalable and efficient in modeling large datasets [12]. It employs advanced techniques such as regularized learning, parallel processing, and optimized algorithms for handling sparse data, making it a popular choice for various applications, including problems involving time series and pattern prediction in large datasets. These characteristics make XGBoost a viable alternative for workload prediction in cloud computing, as it can analyze historical resource usage data, identify trends, and anticipate future demands, contributing to dynamic and efficient infrastructure allocation.

III. RELATED WORK

Research on proactive auto-scaling in cloud computing has leveraged diverse machine learning (ML) techniques to address workload prediction challenges. Below, we categorize key approaches and highlight their contributions and limitations:

A. Neural Networks for Temporal Pattern Capture

Deep learning models, particularly Long Short-Term Memory (LSTM) networks, have been widely adopted for workload forecasting due to their ability to capture temporal dependencies. For instance, [27] proposed an LSTM-based model to predict HTTP request rates in cloud data centers, achieving an improvement in prediction accuracy over ARIMA. Similarly, [28] combined Convolutional Neural Networks (CNNs) with feedforward networks to predict CPU utilization, reducing SLA violations.

B. Reinforcement Learning and Hybrid Approaches

Reinforcement Learning (RL) has been explored to optimize auto-scaling policies. [29] introduced RLPAS, an RL-based framework that dynamically adjusts VM allocations based on real-time metrics, reducing resource waste.

C. Support Vector Machines (SVM) and Regression Models

VM-based approaches have demonstrated robustness in high-dimensional workload prediction. [31] applied SVM regression to forecast server loads in IaaS environments. [30] extended this work with MLscale, a framework combining SVM with regression trees to predict application metrics, reducing provisioning delays by 15%.

While existing studies validate the potential of machine learning for proactive auto-scaling, the use of XGBoost in cloud workload prediction remains largely unexplored. Most approaches either rely on computationally intensive deep learning models [27], [28], which may not be feasible for real-time scalability, or face challenges in handling imbalanced workload data, leading to suboptimal predictions [1]. Exploring XGBoost as an alternative could bridge this gap, offering a more efficient and interpretable solution for workload forecasting in cloud environments.

IV. PROPOSED MODEL

The motivation and objective of this work is to propose a cloud workload prediction model using the XGBoost machine learning library for multi-class classification. The proposed model will be trained to analyze the workload arrival rate across various time intervals and the impact of these workloads on resource utilization in a cloud datacenter. Based on the learning process, the model will be able to predict the trend of future workloads and their respective impact on resource utilization.

A. Extreme Gradient Boost for Multi-Class Classification

Based on the boosting principle [32] [33], XGBoost, when applied to classification problems, iteratively combines multiple decision tree classifiers to address problems involving more than two classes ($c > 2$). Rather than directly separating the classes as binary methods do, XGBoost builds an ensemble of trees that, through successive weighting and updating, minimizes prediction errors at each iteration. This approach enables the model to capture complex relationships among the

variables, accurately distinguishing between different demand levels.

In practice, XGBoost is configured to employ a softmax function in its final layer, which distributes the probabilities among the classes and selects the winning class based on the highest cumulative probability [12]. This methodology not only ensures a reduction in errors through the iterative correction of residuals, but also offers robustness against potential overfitting (biased model), thanks to its internal regularization mechanisms and the update strategy of the "weak learners." Consequently, when applying XGBoost for workload prediction, we obtain classifiers that efficiently adapt to the variations present in the cloud computing environment, thereby promoting proactive and more assertive auto-scaling.

B. Workload Characterization Features

We have employed nine features [34] to characterize the workload for which our model will perform predictions. These features, as outlined in TABLE I, can be easily obtained from any system and serve as key performance indicators of a node in a cloud datacenter.

Since the rate at which jobs arrive directly impacts node performance and the speed of service delivery, job counts collected over time intervals of one, five, and fifteen minutes, along with their relative effects on computing resources and network throughput, are used to train and base our prediction model.

TABLE I
FEATURES OF WORKLOAD [34]

S. No.	Feature Name
1	The number of jobs in a minute
2	The number of jobs in 5 minutes
3	The number of jobs in 15 minutes
4	Memory capacity
5	Disk capacity
6	Number of CPU cores
7	CPU speed per core
8	Average Receive for Network Bandwidth in Kbps
9	Average Transmit for Network Bandwidth in Kbps

V. EXPERIMENTAL SETUP

A. Dataset Details

For the realization of this study, the dataset used is available and was extracted from [1]. The composition of the dataset is presented as follows: nine attributes obtained from the use of 13 cloud nodes. Among all the files available in the dataset [1], the file selected for the experiment in this study contains 25,697 entries. Of these entries, 19,273 (75%) were used for training the model, while 6,424 (25%) were set aside for testing. The responses recorded in the data set are characterized into 4 ordinal classes, as described in TABLE II.

TABLE II
CLASS LABELS WITH RANGE

S. No.	Class Label	Range
1	High	76-100%
2	Medium	51-75%
3	Low	26-50%
4	Very Low	0-25%

B. Implementation Tools

We utilized Google Colab, a hosted Jupyter Notebook service that requires no setup and offers free access to computing resources, including GPUs and TPUs, to program the classification model in Python. The information provided by Colab regarding the notebook specifications is: 12.7 GB of RAM and 107.7 GB of Disk.

C. Metrics for Classifier Evaluation

It is of fundamental and indisputable importance that we measure the performance of machine learning models, as this provides insight into the accuracy with which the model is performing the forecasting task to achieve the desired result. To evaluate the performance of the proposed model, we used a set of metrics, which are presented in TABLE III.

TABLE III
PERFORMANCE METRICS

S. No.	Metric Name
1	Classification Score
2	Balanced Accuracy Score
3	Mean Absolute Error
4	Mean Squared Error
5	Root Mean Squared Error

D. XGBoost Hyperparameter Configuration

For the workload classification task, we employed the *XGBoost* library [12], configuring it to handle multi-class classification with four distinct classes.

- **objective = multi:softmax**: Configures XGBoost for a multi-class classification task, using the *softmax* transformation at the output layer to determine which of the four classes is most likely.
- **num_class**: Specifies the total number of classes (in this case, 4), ensuring that the model correctly distributes probabilities among all classes.
- **eval_metric = mlogloss**: Uses multi-class *log loss* as an internal evaluation metric. This choice is appropriate for scenarios where we strongly penalize incorrect classifications, making the model more sensitive to class imbalances.
- **eta = 0.1**: Also known as the *learning rate*, it controls the weight of each boosting iteration. Lower values help the model learn more gradually, reducing the chance of overfitting.

- **max_depth = 6**: Defines the maximum depth of the decision trees. Deeper trees can capture more complex interactions but may lead to overfitting. Setting this parameter to 6 strikes a balance between complexity and performance.
- **subsample = 0.8**: Determines the fraction of instances (rows) used to build each tree. This sampling method reduces the correlation among trees, helping to counteract overfitting.
- **colsample_bytree = 0.8**: Similar to *subsample*, but it operates on variables (columns). Selecting 80% of the features for each tree increases the diversity of the ensemble.

The *subsample* and *colsample_bytree* settings were fine-tuned to reduce the model's variance and avoid overfitting, while *max_depth* was determined through preliminary experiments to achieve an appropriate balance between bias and variance. Additionally, we fixed *eta* at 0.1 to ensure smoother learning, preventing overly rapid convergence that could lead to error spikes in minority classes. Finally, for the number of boosting rounds (*num_boost_round* = 100), we performed empirical tests ranging from 50 to 300 iterations and observed that values above 100 did not yield significant gains in accuracy, while increasing the risk of overfitting.

The parameter set above provided the best results in our experiments, balancing good classification performance and acceptable processing time. In the following sections, we quantitatively evaluate the model's performance, comparing it to the *Support Vector Machines* (SVM) approach proposed in [2].

VI. RESULTS AND DISCUSSION

To evaluate the proposed workload prediction model, we generated a confusion matrix and compared key metrics, such as *Accuracy*, *Balanced Accuracy*, *Mean Absolute Error* (MAE), and *Root Mean Squared Error* (RMSE), against those obtained using the SVM-based model from [2].

A. Confusion Matrix Analysis

The confusion matrix generated for the XGBoost classifier is presented in Fig. 1. The overall accuracy of the XGBoost model reached **98.23%**, significantly outperforming the SVM classifier with its accuracy of **90%** as presented in [2]. The confusion matrix demonstrates that the XGBoost model effectively classified most instances across all four workload classes.

Notably, the model achieved excellent precision and recall for the *High* and *Very Low* classes, correctly classifying **3276 out of 3280** instances and **2367 out of 2379** instances, respectively. For the *Low* class, the model accurately predicted **634 out of 701** instances, with some misclassifications primarily directed toward the *Very Low* class. The *Medium* class, which presents challenges due to its smaller representation, had **34 out of 65** instances correctly classified, with errors mostly distributed among the *High* and *Low* classes.

These results highlight that while XGBoost achieved high overall performance, improvements could still be made for the *Medium* class, emphasizing the need for further balancing and feature engineering.

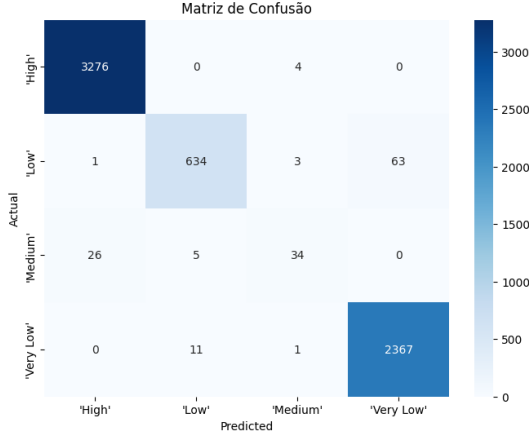


Fig. 1. Confusion Matrix for the XGBoost Classifier.

B. Comparison of Classifiers Based on Evaluation Metrics

Table IV presents a comparison of the performance of the SVM and XGBoost classifiers across key evaluation metrics. The XGBoost model consistently outperformed the SVM model, particularly in reducing error, achieving lower MAE and RMSE values.

TABLE IV
COMPARISON OF CLASSIFIERS BASED ON EVALUATION METRICS

Metric	SVM Classifier [2]	XGBoost Classifier
Balanced Accuracy Score	0.894	0.855
Mean Absolute Error (MAE)	0.180	0.034
Mean Squared Error (MSE)	0.420	0.066
Root Mean Squared Error (RMSE)	0.640	0.257

C. Classification Score Analysis

The classification scores for the SVM classifier are summarized in Table V, as reported in [2]. Table VI presents the corresponding scores for the XGBoost model. The XGBoost classifier demonstrated substantial improvements, particularly in precision and recall for the *Low* and *Medium* classes.

TABLE V
CLASSIFICATION SCORE FOR SVM CLASSIFIER [2]

Class	Precision	Recall	F1-Score	Support
High	0.99	1.00	0.99	3280
Low	0.56	0.90	0.69	701
Medium	0.00	0.00	0.00	65
Very Low	0.89	0.87	0.88	2379

TABLE VI
CLASSIFICATION SCORE FOR XGBOOST CLASSIFIER

Class	Precision	Recall	F1-Score	Support
High	0.99	1.00	1.00	3280
Low	0.98	0.90	0.94	701
Medium	0.85	0.52	0.64	65
Very Low	0.98	0.99	0.98	2379

D. Discussion

The results demonstrate that the XGBoost classifier consistently outperforms the SVM-based model in terms of overall error reduction and accurate classification of key classes. Although the SVM classifier achieves a higher balanced accuracy score (0.894 compared to 0.855), the XGBoost model compensates with significantly lower MAE and RMSE, making it more suitable for scenarios where prediction errors need to be minimized.

The XGBoost classifier excels in the classification of the *Low* and *Medium* classes, which are often challenging in imbalanced datasets. With precision improvements for these minority classes, XGBoost shows robustness in handling complex data distributions. However, the challenges posed by the *Medium* class indicate potential areas for future optimization, possibly involving further tuning or data augmentation strategies.

The classification model based on XGBoost demonstrates enhanced performance when handling a dataset in which values belonging to a single class occur with great frequency. In light of this, and after analyzing the Confusion Matrix 1, we were able to understand the difficulty of the model in forecasting data for the *Medium* class. In a real-world context, specifically for a node operating in the cloud, this issue may occur only rarely, however, the same should be treated and needs attention.

These findings emphasize the importance of using multiple evaluation metrics when selecting machine learning models for workload prediction. Although SVM showed slightly better balance between classes, the superior error minimization and overall classification performance of XGBoost highlight its practical advantage in proactive resource provisioning scenarios within cloud environments.

VII. CONCLUSION AND FUTURE WORK

In this paper, we explored the potential of XGBoost as an alternative to SVM for proactive workload prediction in cloud environments. By employing real-world data from [1] and [2], our experimental results demonstrate that XGBoost is capable of significantly reducing forecasting errors while maintaining high classification accuracy. Specifically, the XGBoost model achieves an accuracy of **98.23%**—a marked improvement over the **90%** accuracy reported for the SVM-based approach—along with substantially lower MAE and RMSE values.

Although XGBoost proved notably effective, particularly for the *Low* and *Very Low* classes, some misclassifications

persist for the *Medium* class, underscoring the importance of further exploration in data balancing and feature engineering. Future work may therefore focus on augmenting minority classes, tuning hyperparameters, and incorporating additional contextual features (e.g., network latencies, geographical data, or application-level metrics) to improve the model's generalization and address any remaining imbalances.

Moreover, extending the proposed framework to real-time or near-real-time deployments offers a practical next step. Implementing streaming data pipelines and online learning methods could enable more responsive, proactive auto-scaling policies. Such enhancements would further reduce SLA violations and bolster Quality of Service (QoS) in dynamic cloud settings. Overall, our findings reinforce that gradient boosting methods—exemplified by XGBoost—can serve as robust, scalable solutions for workload forecasting, thereby driving more intelligent and cost-effective resource management in modern cloud infrastructures.

REFERENCES

- [1] AL-FAIFI, Abdullah Mohammed et al. Data on performance prediction for cloud service selection. *Data in brief*, v. 20, p. 1039-1043, 2018.
- [2] SINGH, Sanjay T.; TIWARI, Mahendra; DHAR, Anchit Sajal. Machine Learning based Workload Prediction for Auto-scaling Cloud Applications. In: 2022 OPJU International Technology Conference on Emerging Technologies for Sustainable Development (OTCON). IEEE, 2023. p. 1-6.
- [3] AL-DHURAIBI, Y. et al. Elasticity in Cloud Computing: State of the Art and Research Challenges. *IEEE Transactions on Services Computing*, v. 11, n. 2, p. 430-447, 2018.
- [4] HERBST, N. R.; KOUNEV, S.; REUSSNER, R. Elasticity in Cloud Computing: What It Is, and What It Is Not. In: 10th International Conference on Autonomic Computing (ICAC '13), 2013, San Jose, CA. Proceedings [...]. USENIX Association, 2013. p. 23-27.
- [5] ABDELMABOUD, A. et al. Quality of service approaches in cloud computing: A systematic mapping study. *Journal of Systems and Software*, v. 101, p. 159-179, 2015.
- [6] PATEL, P.; RANABAHU, A. H.; SHETH, A. P. Service Level Agreement in Cloud Computing. In: Cloud Workshops at OOPSLA09, 2009, Orlando, FL. Proceedings [...]. ACM, 2009.
- [7] CHEN, T. et al. Cloud workload prediction using machine learning model based on rough set theory. *IEEE Transactions on Cloud Computing*, v. 9, n. 3, p. 1228-1240, 2021.
- [8] JIANG, J. et al. Proactive and reactive auto-scaling mechanisms for cloud-based systems: A systematic literature review. *Cluster Computing*, v. 24, n. 3, p. 2687-2716, 2021.
- [9] LORIDO-BOTRAN, T.; MIGUEL-ALONSO, J.; LOZANO, J. A. A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments. *Journal of Grid Computing*, v. 12, n. 4, p. 559-592, 2014.
- [10] CHEN, B.; CHANG, M.; LIN, C. Load Forecasting Using Support Vector Machines: A Study on EUNITE Competition 2001. *IEEE Transactions on Power Systems*, v. 19, n. 4, p. 1821-1830, 2004.
- [11] ZHANG, Q. et al. A Deep Learning Approach for VM Workload Prediction in the Cloud. In: 2018 IEEE 24th International Conference on Parallel and Distributed Systems (ICPADS), 2018, Singapore. Proceedings [...]. IEEE, 2018. p. 82-89.
- [12] CHEN, T.; GUESTRIN, C. XGBoost: A Scalable Tree Boosting System. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, San Francisco, CA. Proceedings [...]. ACM, 2016. p. 785-794.
- [13] GEORGANOS, S. et al. Very High Resolution Object-Based Land Use–Land Cover Urban Classification Using Extreme Gradient Boosting. *IEEE Geoscience and Remote Sensing Letters*, v. 15, n. 4, p. 607-611, 2018.
- [14] ISLAM, Rafia et al. The future of cloud computing: benefits and challenges. *International Journal of Communications, Network and System Sciences*, v. 16, n. 4, p. 53-65, 2023.
- [15] AZMANDIAN, Fatemeh et al. Workload characterization at the virtualization layer. In: 2011 IEEE 19th annual international symposium on modelling, analysis, and simulation of computer and telecommunication systems. IEEE, 2011. p. 63-72.
- [16] JING, Xue; JIAN-JUN, Zhang. A brief survey on the security model of cloud computing. In: 2010 ninth international symposium on distributed computing and applications to business, engineering and science. IEEE, 2010. p. 475-478.
- [17] AL-DHURAIBI, Yahya et al. Elasticity in cloud computing: state of the art and research challenges. *IEEE Transactions on services computing*, v. 11, n. 2, p. 430-447, 2017.
- [18] QU, Chenhao; CALHEIROS, Rodrigo N.; BUYYA, Rajkumar. Auto-scaling web applications in clouds: A taxonomy and survey. *ACM Computing Surveys (CSUR)*, v. 51, n. 4, p. 1-33, 2018.
- [19] ALHARTHI, Saleha et al. Auto-Scaling Techniques in Cloud Computing: Issues and Research Directions. *Sensors*, v. 24, n. 17, p. 5551, 2024.
- [20] LORIDO-BOTRAN, Tania; MIGUEL-ALONSO, Jose; LOZANO, Jose A. A review of auto-scaling techniques for elastic applications in cloud environments. *Journal of grid computing*, v. 12, p. 559-592, 2014.
- [21] SAXENA, Deepika; SINGH, Ashutosh Kumar. Workload forecasting and resource management models based on machine learning for cloud computing environments. *arXiv preprint arXiv:2106.15112*, 2021.
- [22] KUMAR, Jitendra; SINGH, Ashutosh Kumar. Workload prediction in cloud using artificial neural network and adaptive differential evolution. *Future Generation Computer Systems*, v. 81, p. 41-52, 2018.
- [23] FENG, Binbin; DING, Zhijun. Application-Oriented Cloud Workload Prediction: A Survey and New Perspectives. *Tsinghua Science and Technology*, v. 30, n. 1, p. 34-54, 2024.
- [24] CALZAROSSA, Maria Carla et al. Workloads in the Clouds. *Principles of Performance and Reliability Modeling and Evaluation: Essays in Honor of Kishor Trivedi on his 70th Birthday*, p. 525-550, 2016.
- [25] KHAN, Tahseen et al. Machine learning (ML)-centric resource management in cloud computing: A review and future directions. *Journal of Network and Computer Applications*, v. 204, p. 103405, 2022.
- [26] SAXENA, Deepika et al. Performance analysis of machine learning centered workload prediction models for cloud. *IEEE Transactions on Parallel and Distributed Systems*, v. 34, n. 4, p. 1313-1330, 2023.
- [27] KUMAR, Jitendra; GOOMER, Rimsha; SINGH, Ashutosh Kumar. Long short term memory recurrent neural network (LSTM-RNN) based workload forecasting model for cloud datacenters. *Procedia computer science*, v. 125, p. 676-682, 2018.
- [28] GOLSHANI, Ehsan; ASHTIANI, Mehrdad. Proactive auto-scaling for cloud environments using temporal convolutional neural networks. *Journal of Parallel and Distributed Computing*, v. 154, p. 119-141, 2021.
- [29] BIBAL BENIFA, J. V.; DEJEY, D. Rlpas: Reinforcement learning-based proactive auto-scaler for resource provisioning in cloud environment. *Mobile Networks and Applications*, v. 24, p. 1348-1363, 2019.
- [30] WAJAHAT, Muhammad et al. Mlscale: A machine learning based application-agnostic autoscaler. *Sustainable Computing: Informatics and Systems*, v. 22, p. 287-299, 2019.
- [31] MORENO-VOZMEDIANO, Rafael et al. Efficient resource provisioning for elastic cloud services based on machine learning techniques. *Journal of Cloud Computing*, v. 8, n. 1, p. 1-18, 2019.
- [32] FREUND, Y.; SCHAPIRE, R. E. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, v. 55, n. 1, p. 119-139, 1997.
- [33] FRIEDMAN, J. H. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, v. 29, n. 5, p. 1189-1232, 2001.
- [34] A. M. Al-Faifi, B. Song, M. M. Hassan, A. Alamri, and A. Gumaiei, "Performance prediction model for cloud service selection from smart data," *Futur. Gener. Comput. Syst.*, vol. 85, pp. 97–106, 2018, doi: 10.1016/j.future.2018.03.015.